Python Niveau I

Solutions complètes — Python Niveau I

Jour 1

Module 1 : Premiers pas avec Python

Exercice 1.1 : La console comme calculatrice

```
45 + 32        # 77
256 / 8        # 32.0
2 ** 10        # 1024
17 % 5         # 2
17 // 5        # 3
```

Exercice 1.2 : Créer et manipuler des variables

```
prix_ht = 85.50
tva = 0.20
prix_ttc = prix_ht * (1 + tva)
print(f"Le prix TTC est de {prix_ttc} euros")
print(type(prix_ht), type(tva), type(prix_ttc))
```

Exercice 1.3 : Écrire un script complet

```
from math import pi

rayon = 7
perimetre = 2 * pi * rayon
aire = pi * rayon ** 2
print(f"Perimetre : {perimetre:.2f}")  # 43.98
print(f"Aire : {aire:.2f}")            # 153.94
```

Exercice 1.4 : Utiliser l'aide

```
help(round)
print(round(3.14159265, 2))   # 3.14
```

```
print(round(3.14159265, 4))   # 3.1416
print(type(42), type(3.14), type("bonjour"), type(True))
print([m for m in dir("texte") if not m.startswith("_")])
```

Exercice 1.5 : Environnement de travail

```
import sys, math, datetime

print(sys.version)
print(f"pi = {math.pi}, e = {math.e}")
print(math.sqrt(144), math.factorial(6))
print(datetime.date.today())
print([f for f in dir(math) if not f.startswith("_")])
```

Module 2 : Structures de données

Exercice 2.1 : Types fondamentaux

```
x_int, x_float, x_str, x_bool = 42, 3.14, "Bonjour", True
print(type(x_int), type(x_float), type(x_str), type(x_bool))
print(isinstance(x_int, int), isinstance(x_float, float))
print(int("42"), float("3.14"), str(100), bool(0), bool(1), bool(""), bool(
```

Exercice 2.2 : Création de listes

```
l1 = list(range(1, 21))
l2 = list(range(2, 31, 2))
l3 = [0] * 5
l4 = [x ** 2 for x in range(1, 11)]
l5 = [4, 8, 15, 16, 23, 42]
for nom, lst in [("l1",l1),("l2",l2),("l3",l3),("l4",l4),("l5",l5)]:
    print(f"{nom} ({len(lst)}) = {lst}")
```

Exercice 2.3 : Indexation et slicing

```
notes = [12, 15, 8, 17, 14, 9, 11, 16, 7, 13]
print(notes[2])       # 8
print(notes[-1])      # 13
print(notes[1:5])     # [15, 8, 17, 14]
print(notes[::2])     # [12, 8, 14, 11, 7]
```

```
print(notes[::-1])    # liste inversee
print(notes[-3:])     # [16, 7, 13]
```

Exercice 2.4 : Opérations sur listes

```
fruits = ["pomme", "banane", "orange"]
fruits.append("fraise")
fruits.insert(1, "kiwi")
fruits.extend(["mangue", "ananas"])
fruits.remove("banane")
fruits.sort()
fruits.reverse()
print(fruits)
```

Exercice 2.5 : Tuples

```
coordonnees = (48.8566, 2.3522)
lat, lon = coordonnees
villes = (("Paris", 48.8566, 2.3522), ("Lyon", 45.7640, 4.8357), ("Marseill
for nom, lat, lon in villes:
    print(f"{nom} : {lat}, {lon}")
try:
    coordonnees[0] = 0
except TypeError as e:
    print(f"Erreur : {e}")
distances = {("Paris", "Lyon"): 465, ("Paris", "Marseille"): 775}
print(distances[("Paris", "Lyon")])
```

Exercice 2.6 : Dictionnaires

```
etudiant = {"nom": "Dupont", "prenom": "Marie", "age": 22, "notes": [14, 16
print(etudiant["prenom"], etudiant.get("prenom"))
etudiant["email"] = "marie.dupont@univ.fr"
print(list(etudiant.keys()))
print(list(etudiant.values()))
for cle, valeur in etudiant.items():
    print(f"  {cle} : {valeur}")
print(etudiant.get("telephone", "Non renseigne"))
```

Exercice 2.7 : Ensembles (sets)

```
a, b = {1, 2, 3, 4, 5}, {4, 5, 6, 7, 8}
print(a | b)      # union
print(a & b)      # intersection
```

```
print(a - b)      # difference
print(a ^ b)      # difference symetrique
print(3 in a)     # True
print(set([1, 2, 2, 3, 3, 3]))  # {1, 2, 3}
```

Exercice 2.8 : Conversions et mutabilité

```
ma_liste = [1, 2, 3, 4, 5]
mon_tuple = tuple(ma_liste)
mon_set = set(mon_tuple)
ma_liste[0] = 99  # OK
try:
    mon_tuple[0] = 99
except TypeError as e:
    print(e)
a = [1, 2, 3]
b = a           # reference
c = a.copy()    # copie
a[0] = 999
print(b)  # [999, 2, 3] — modifie
print(c)  # [1, 2, 3]   — intact
```

Module 3 : Fonctions et contrôle

Exercice 3.1 : Conditions if/elif/else

```
age = 25
if age < 18:      categorie = "Mineur"
elif age < 30:    categorie = "Jeune adulte"
elif age < 60:    categorie = "Adulte"
else:             categorie = "Senior"
print(f"Age {age} : {categorie}")

note = 14
if   note >= 16: mention = "Excellent"
elif note >= 14: mention = "Bien"
elif note >= 12: mention = "Assez bien"
elif note >= 10: mention = "Passable"
else:            mention = "Insuffisant"
print(f"Note {note} : {mention}")
```

Exercice 3.2 : Boucles for

```
for i in range(1, 11):
    print(i, end=" ")
print()

fruits = ["pomme", "banane", "orange", "fraise"]
for i, fruit in enumerate(fruits):
    print(f"{i} : {fruit}")

total = sum(range(1, 101))
print(f"Somme 1-100 : {total}")

for cle, val in {"a": 1, "b": 2, "c": 3}.items():
    print(f"{cle} -> {val}")
```

Exercice 3.3 : Boucles while

```
import random

n = 10
while n >= 0:
    print(n, end=" ")
    n -= 1
print("Decollage !")

n, fact, i = 7, 1, 1
while i <= n:
    fact *= i; i += 1
print(f"{n}! = {fact}")

n = 0
while 2 ** n <= 1000:
    n += 1
print(f"n = {n}, 2^{n} = {2**n}")

random.seed(42)
de, lancers = 0, 0
while de != 6:
    de = random.randint(1, 6)
    lancers += 1
print(f"{lancers} lancers pour obtenir un 6")
```

Exercice 3.4 : Fonctions simples

```
def celsius_to_fahrenheit(c):
    return c * 9 / 5 + 32
```

```
def imc(poids, taille):
    return round(poids / taille ** 2, 2)

def est_premier(n):
    if n < 2: return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0: return False
    return True

print(celsius_to_fahrenheit(0), celsius_to_fahrenheit(100))
print(imc(75, 1.80))
print([n for n in range(1, 21) if est_premier(n)])
```

**Exercice 3.5 : Arguments par défaut et args*

```
def saluer(nom, salutation="Bonjour"):
    return f"{salutation}, {nom} !"

def puissance(base, exposant=2):
    return base ** exposant

def moyenne(*args):
    return sum(args) / len(args) if args else 0

def fiche(**kwargs):
    for cle, val in kwargs.items():
        print(f"  {cle} : {val}")

print(saluer("Marie"))
print(puissance(5), puissance(2, 10))
print(moyenne(10, 20, 30))
fiche(nom="Dupont", prenom="Marie", age=22)
```

Exercice 3.6 : Compréhensions de liste

```
carres = [x ** 2 for x in range(1, 16)]
pairs = [x for x in range(1, 31) if x % 2 == 0]
mots = ["Python", "est", "un", "langage", "puissant"]
longueurs = [len(m) for m in mots]
mots_longs = [m for m in mots if len(m) > 3]
paires = [(x, x ** 2) for x in range(1, 11)]
plat = [e for sous in [[1,2],[3,4],[5,6]] for e in sous]
print(carres, pairs, longueurs, mots_longs, paires, plat, sep="\n")
```

Exercice 3.7 : Fonctions avancées

```
carre = lambda x: x ** 2
print(carre(5))

celsius = [0, 10, 20, 25, 30, 37, 100]
fahrenheit = list(map(lambda c: c * 9/5 + 32, celsius))
print(fahrenheit)

nombres = [-3, 7, -1, 4, -8, 2, 0, 5]
positifs = list(filter(lambda x: x > 0, nombres))
print(positifs)

personnes = [("Alice", 30), ("Bob", 22), ("Clara", 27), ("David", 19)]
par_age = sorted(personnes, key=lambda p: p[1])
print(par_age)

pairs_carres = list(map(lambda x: x**2, filter(lambda x: x%2==0, range(1,21
print(pairs_carres)

mots = ["banane", "pomme", "kiwi", "fraise", "ananas"]
print(sorted(mots, key=lambda m: m[-1]))
```

Jour 2

Module 5 : Fichiers et modules

Exercice 5.1 : Lire un fichier texte

```
with open("villes.txt", "w") as f:
    for v in ["Paris", "Lyon", "Marseille", "Toulouse", "Bordeaux"]:
        f.write(v + "\n")

with open("villes.txt", "r") as f:
    print(f.read())

with open("villes.txt", "r") as f:
    villes = [l.strip() for l in f]
print(villes)
```

Exercice 5.2 : Écrire dans un fichier

```
with open("courses.txt", "w") as f:
    for a in ["pain", "lait", "beurre", "oeufs", "fromage"]:
        f.write(a + "\n")
with open("courses.txt", "a") as f:
    f.write("pommes\nchocolat\n")
with open("courses.txt") as f:
    for i, l in enumerate(f, 1):
        print(f"{i}. {l.strip()}")
```

Exercice 5.3 : Manipuler des CSV

```
import csv

with open("employes.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["nom", "poste", "salaire"])
    w.writerows([["Dupont","Dev",45000],["Martin","Design",40000],
                 ["Bernard","Chef",52000],["Petit","Analyste",42000],["Robe

with open("employes.csv") as f:
    for row in csv.DictReader(f):
        print(f"{row['nom']} : {row['salaire']} EUR")

with open("employes.csv") as f:
    salaires = [int(r["salaire"]) for r in csv.DictReader(f)]
print(f"Moyenne : {sum(salaires)/len(salaires):.0f} EUR")
```

Exercice 5.4 : Manipuler du JSON

```
import json

produit = {"nom": "Laptop Pro", "prix": 1299.99, "stock": 42,
           "categories": ["informatique", "bureautique"],
           "specs": {"ram": "16 Go", "stockage": "512 Go SSD"}}

with open("produit.json", "w") as f:
    json.dump(produit, f, indent=2, ensure_ascii=False)

with open("produit.json") as f:
    data = json.load(f)
print(data["nom"], data["prix"])

chaine = json.dumps(produit, indent=2, ensure_ascii=False)
```

```
ville = json.loads('{"ville": "Paris", "population": 2161000}')
print(ville)
```

Exercice 5.5 : Explorer le système de fichiers

```
import os

print(os.getcwd())
print(os.listdir(".")[:10])
print(os.path.exists("produit.json"))
print(os.path.getsize("produit.json"))

for nom in ["rapport.pdf", "photo.jpg", "script.py"]:
    base, ext = os.path.splitext(nom)
    print(f"{nom} -> base='{base}', ext='{ext}'")

print(os.path.join("dossier", "sous-dossier", "fichier.txt"))
```

Exercice 5.6 : Importer et utiliser des modules

```
import math
from datetime import date
from collections import Counter

print(math.sqrt(144), math.ceil(3.2), math.floor(3.8))
print(date.today())
compteur = Counter(["pomme","banane","pomme","orange","banane","pomme"])
print(compteur.most_common(2))
```

Module 6 : NumPy et pandas

Exercice 6.1 : NumPy — Créer des arrays

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
b = np.arange(20)
c = np.linspace(0, 1, 10)
zeros = np.zeros((3, 4))
uns = np.ones((3, 4))
identite = np.eye(4)
```

```
for nom, arr in [("a",a),("b",b),("c",c),("zeros",zeros)]:
    print(f"{nom}: dtype={arr.dtype}, shape={arr.shape}, ndim={arr.ndim}")
```

Exercice 6.2 : NumPy — Opérations vectorisées

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])
b = np.array([1, 2, 3, 4, 5])
print(a + 5, a * 2, a ** 2)
print(a + b, a * b, a / b)
print(a[a > 25])
print(f"mean={a.mean()}, std={a.std():.2f}, sum={a.sum()}")
print(f"cumsum={a.cumsum()}")

np.random.seed(42)
data = np.random.randn(1000)
print(f"Normal: mean={data.mean():.4f}, std={data.std():.4f}")
```

Exercice 6.3 : Créer un DataFrame

```
import pandas as pd

df = pd.DataFrame({"nom": ["Alice","Bob","Clara","David","Emma"],
                   "age": [28,35,22,41,30], "ville": ["Paris","Lyon","Pari
                   "salaire": [45000,52000,38000,61000,48000]})
print(df.head(3))
print(df.dtypes)
df = df.rename(columns={"salaire": "salaire_annuel"})
```

Exercice 6.4 : Charger et explorer

```
import pandas as pd
import seaborn as sns

tips = sns.load_dataset("tips")
print(tips.head())
tips.info()
print(tips.describe())
print(f"Shape : {tips.shape}")
print(tips.columns.tolist())
```

Exercice 6.5 : Sélection et filtrage

```
import pandas as pd, seaborn as sns
tips = sns.load_dataset("tips")

bills = tips["total_bill"]
subset = tips[["total_bill", "tip"]]
print(tips.iloc[10:16])
print(tips[tips["total_bill"] > 40])
print(tips[(tips["day"] == "Sun") & (tips["tip"] > 5)])
print(tips.query("day.isin(['Sat','Sun']) and total_bill > 30"))
print(tips[tips["day"].isin(["Sat", "Sun"])].shape[0])
```

Exercice 6.6 : Tri et colonnes calculées

```
import pandas as pd, seaborn as sns
tips = sns.load_dataset("tips")

print(tips.sort_values("total_bill", ascending=False).head())
tips["tip_pct"] = (tips["tip"] / tips["total_bill"] * 100).round(1)
tips["categorie"] = tips["total_bill"].apply(
    lambda x: "Petit" if x < 15 else ("Moyen" if x < 30 else "Gros"))
jours_fr = {"Sun":"Dimanche","Sat":"Samedi","Thur":"Jeudi","Fri":"Vendredi"
tips["jour_fr"] = tips["day"].map(jours_fr)
print(tips["categorie"].value_counts())
```

Exercice 6.7 : Merge et concat

```
import pandas as pd

employes = pd.DataFrame({"id":[1,2,3,4,5], "nom":["Alice","Bob","Clara","Da
                         "dept_id":[10,20,10,30,99]})
departements = pd.DataFrame({"dept_id":[10,20,30,40], "dept_nom":["IT","Mar

print(pd.merge(employes, departements, on="dept_id", how="inner"))
print(pd.merge(employes, departements, on="dept_id", how="left"))
print(pd.merge(employes, departements, on="dept_id", how="outer"))

q1 = pd.DataFrame({"mois":["Jan","Fev","Mar"], "ca":[100,120,110]})
q2 = pd.DataFrame({"mois":["Avr","Mai","Jun"], "ca":[130,115,140]})
print(pd.concat([q1, q2], ignore_index=True))
```

Exercice 6.8 : Groupby et agrégation

```
import pandas as pd, seaborn as sns
tips = sns.load_dataset("tips")
```

```
print(tips.groupby("day")["tip"].mean().round(2))
print(tips.groupby("day").agg(tip_mean=("tip","mean"), tip_sum=("tip","sum"
                              count=("total_bill","count")).round(2))
print(tips.groupby(["day","time"])["total_bill"].mean().round(2))

tips["tip_moy_jour"] = tips.groupby("day")["tip"].transform("mean").round(2

pivot = pd.pivot_table(tips, values="tip", index="day", columns="time", agg
print(pivot)
```

Module 7 : Visualisation

Exercice 7.1 : Nuage de points

```
import matplotlib.pyplot as plt, seaborn as sns, numpy as np
tips = sns.load_dataset("tips")

fig, ax = plt.subplots(figsize=(8, 6))
couleurs = {"Male": "#2196F3", "Female": "#E91E63"}
for sexe, grp in tips.groupby("sex"):
    ax.scatter(grp["total_bill"], grp["tip"], c=couleurs[sexe],
               s=grp["size"]*20, alpha=0.6, label=sexe, edgecolors="white")
coeffs = np.polyfit(tips["total_bill"], tips["tip"], 1)
x_line = np.linspace(tips["total_bill"].min(), tips["total_bill"].max(), 10
ax.plot(x_line, np.poly1d(coeffs)(x_line), "k--", label="Regression")
ax.set_xlabel("Addition ($)"); ax.set_ylabel("Pourboire ($)")
ax.set_title("Addition vs Pourboire"); ax.legend()
plt.tight_layout(); plt.savefig("scatter.png", dpi=150); plt.show()
```

Exercice 7.2 : Histogramme

```
import matplotlib.pyplot as plt, seaborn as sns
tips = sns.load_dataset("tips")

fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(tips["total_bill"], bins=20, color="#42A5F5", edgecolor="white")
moy, med = tips["total_bill"].mean(), tips["total_bill"].median()
ax.axvline(moy, color="red", linestyle="--", label=f"Moyenne : {moy:.2f}")
ax.axvline(med, color="green", linestyle="-.", label=f"Mediane : {med:.2f}"
ax.set_xlabel("Addition ($)"); ax.set_ylabel("Frequence")
ax.set_title("Distribution des additions"); ax.legend()
plt.tight_layout(); plt.savefig("histogramme.png", dpi=150); plt.show()
```

Exercice 7.3 : Boxplot

```
import matplotlib.pyplot as plt, seaborn as sns
tips = sns.load_dataset("tips")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
jours = ["Thur", "Fri", "Sat", "Sun"]
data = [tips[tips["day"] == j]["tip"].values for j in jours]

bp = axes[0].boxplot(data, labels=["Jeu","Ven","Sam","Dim"], patch_artist=T
                     medianprops=dict(color="red", linewidth=2))
for p, c in zip(bp["boxes"], ["#BBDEFB","#C8E6C9","#FFE0B2","#F8BBD0"]):
    p.set_facecolor(c)
axes[0].set_ylabel("Pourboire ($)"); axes[0].set_title("Pourboires par jour

data_t = [tips[tips["time"]==t]["total_bill"].values for t in ["Lunch","Dir
axes[1].boxplot(data_t, labels=["Dejeuner","Diner"], vert=False, patch_arti
axes[1].set_xlabel("Addition ($)"); axes[1].set_title("Additions par moment
plt.tight_layout(); plt.savefig("boxplot.png", dpi=150); plt.show()
```

Exercice 7.4 : Barplot

```
import matplotlib.pyplot as plt, seaborn as sns, numpy as np
tips = sns.load_dataset("tips")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
jours = ["Thur", "Fri", "Sat", "Sun"]
jours_fr = ["Jeu", "Ven", "Sam", "Dim"]
moy = [tips[tips["day"]==j]["tip"].mean() for j in jours]

bars = axes[0].bar(jours_fr, moy, color=["#42A5F5","#66BB6A","#FFA726","#EF
for b, v in zip(bars, moy):
    axes[0].text(b.get_x()+b.get_width()/2, b.get_height()+0.05, f"{v:.2f}"
axes[0].set_ylabel("Tip moyen ($)"); axes[0].set_title("Tip moyen par jour"

pivot = tips.groupby(["day","time"])["tip"].mean().unstack(fill_value=0).re
x, w = np.arange(4), 0.35
axes[1].bar(x-w/2, pivot.get("Lunch",[0]*4), w, label="Dejeuner", color="#4
axes[1].bar(x+w/2, pivot.get("Dinner",[0]*4), w, label="Diner", color="#EF5
axes[1].set_xticks(x); axes[1].set_xticklabels(jours_fr)
axes[1].set_ylabel("Tip moyen ($)"); axes[1].set_title("Par jour + moment")
plt.tight_layout(); plt.savefig("barplot.png", dpi=150); plt.show()
```

Exercice 7.5 : Multi-graphiques

```
import matplotlib.pyplot as plt, seaborn as sns, numpy as np
tips = sns.load_dataset("tips")

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle("Dashboard Tips", fontsize=16, fontweight="bold")

# Scatter
ax = axes[0,0]
ax.scatter(tips["total_bill"], tips["tip"], alpha=0.5, c="#2196F3", edgecol
c = np.polyfit(tips["total_bill"], tips["tip"], 1)
ax.plot(np.linspace(0,55,100), np.poly1d(c)(np.linspace(0,55,100)), "r--")
ax.set_xlabel("Addition ($)"); ax.set_ylabel("Tip ($)"); ax.set_title("Bill

# Histogramme
ax = axes[0,1]
ax.hist(tips["total_bill"], bins=20, color="#66BB6A", edgecolor="white")
ax.axvline(tips["total_bill"].mean(), color="red", linestyle="--", label="M
ax.set_title("Distribution"); ax.legend()

# Boxplot
ax = axes[1,0]
jours = ["Thur","Fri","Sat","Sun"]
bp = ax.boxplot([tips[tips["day"]==j]["tip"].values for j in jours],
                labels=["Jeu","Ven","Sam","Dim"], patch_artist=True)
for p, col in zip(bp["boxes"], ["#BBDEFB","#C8E6C9","#FFE0B2","#F8BBD0"]):
    p.set_facecolor(col)
ax.set_title("Tip par jour")

# Barplot
ax = axes[1,1]
compte = tips["day"].value_counts().reindex(jours)
ax.bar(["Jeu","Ven","Sam","Dim"], compte.values, color=["#42A5F5","#66BB6A"
ax.set_title("Repas par jour")

plt.tight_layout(); plt.savefig("dashboard.png", dpi=150); plt.show()
```

Exercice 7.6 : Personnalisation avancée

```
import matplotlib.pyplot as plt, seaborn as sns, numpy as np
tips = sns.load_dataset("tips")

stats = tips.groupby("size")["tip"].agg(["mean","std","count"]).reset_index
fig, ax = plt.subplots(figsize=(10, 6))

ax.plot(stats["size"], stats["mean"], "o-", color="#1565C0", lw=2.5, ms=8,
ax.fill_between(stats["size"], stats["mean"]-stats["std"], stats["mean"]+st
                alpha=0.2, color="#42A5F5", label="+/- 1 sd")
```

```
idx = stats["mean"].idxmax()
ax.annotate(f"Max : {stats.loc[idx,'mean']:.2f} $\n(table de {stats.loc[idx
            xy=(stats.loc[idx,"size"], stats.loc[idx,"mean"]),
            xytext=(stats.loc[idx,"size"]+0.8, stats.loc[idx,"mean"]+0.5),
            arrowprops=dict(arrowstyle="->"), bbox=dict(boxstyle="round", f

for _, r in stats.iterrows():
    ax.text(r["size"], r["mean"]-0.35, f"n={int(r['count'])}", ha="center",

ax.set_xlabel("Taille de la table"); ax.set_ylabel("Pourboire moyen ($)")
ax.set_title("Pourboire selon la taille de la table", fontweight="bold")
ax.grid(True, alpha=0.3, linestyle="--"); ax.legend()
plt.tight_layout(); plt.savefig("graphique_pro.png", dpi=200); plt.show()
```